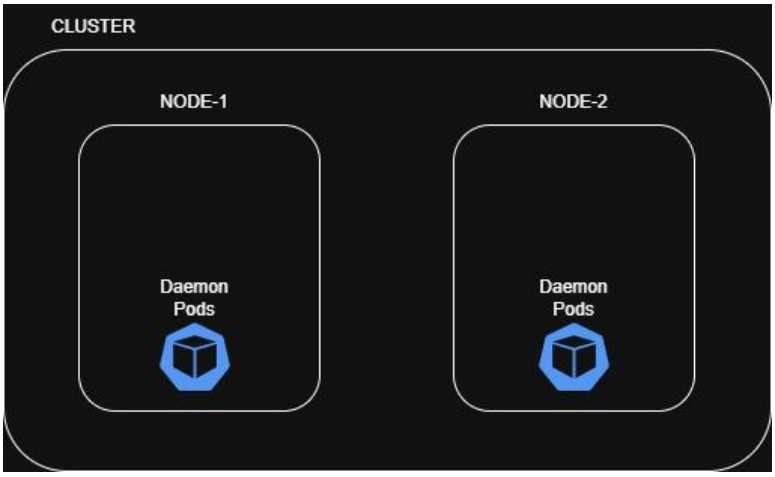
DEPLOYMENTS / STATEFULSETS & DAEMONSET

DAEMONSET

Daemonset is a pod which run continuously on every node to monitor the activity of the node.

Why we use daemonset

1. log collection
2. monitoring
3. networking
4. security agents



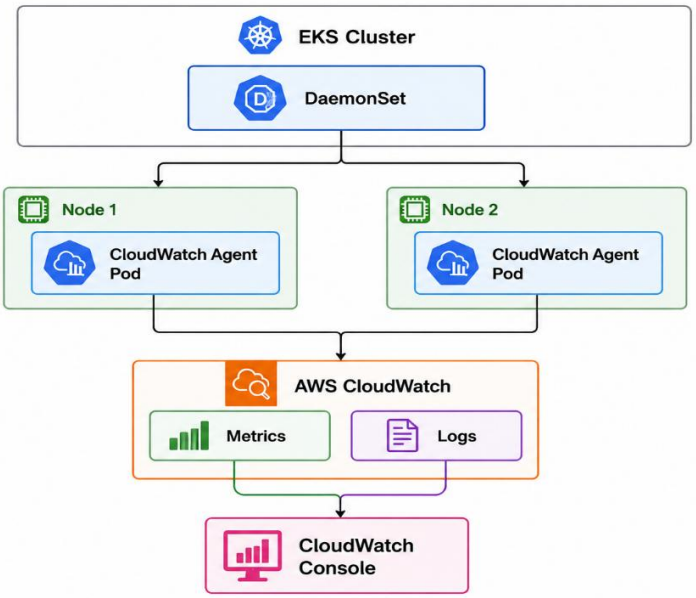
1. vim daemonset.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: my-daemonset
  labels:
    app: myapp
spec:
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
```

- name: nginx-container
  image: amazon/cloudwatch-agent

2. kubectl apply -f daemonset.yaml
3. kubectl get pods

DAEMONSET WITH CLOUDWATCH



1.create a namespace

1. kubectl create namespace daemonset-demo
2. kubectl get ns

2.create a daemonset

1. vim daemonset.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: cloudwatch-demo-daemonset
  namespace: daemonset-demo

spec:
  selector:
    matchLabels:
      app: cloudwatch-demo
  template:
    metadata:
      labels:
        app: cloudwatch-demo
    spec:
      containers:
        - name: demo-container
```

```
      image: nginx:latest
      ports:
        - containerPort: 80
```

2. kubectl apply -f daemonset.yaml
3. kubectl get daemonset -n daemonset-demo
4. kubectl get pods -o daemonset-demo

3.Check Addons

1. aws eks list-addons --cluster-name eks-cluster
2. aws eks create-addon --cluster-name eks-cluster --addon-name eks-pod-identity-agent
3. kubectl get pods -n kube-system

4.create a IAM role

```
aws iam create-role \
  --role-name EKS-CloudWatch-Agent-Role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Principal": {
          "Service": "pods.eks.amazonaws.com"
        },
        "Action": [
          "sts:AssumeRole",
          "sts:TagSession"
        ]
      }
    ]
  }'
```

5.attach CloudWatch policy

```
aws iam attach-role-policy \
  --role-name EKS-CloudWatch-Agent-Role \
  --policy-arn arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
```

6.Verify

```
aws iam list-attached-role-policies \
  --role-name EKS-CloudWatch-Agent-Role
```

7.Get your AWS account ID

```
aws sts get-caller-identity \
  --query Account \
  --output text
```

8.Create pod identity Association

```
aws eks create-pod-identity-association \
  --cluster-name eks-cluster \
```

```
  --namespace amazon-cloudwatch \
  --service-account cloudwatch-agent \
  --role-arn arn:aws:iam::YOUR_ACCOUNT_ID:role/EKS-CloudWatch-Agent-Role
```

{ Change Your Account ID }

9.Install CloudWatch Observability Add-on (now Install)

```
aws eks create-addon \
  --cluster-name eks-cluster \
  --addon-name amazon-cloudwatch-observability
```

10.Check Add-on Status

```
aws eks describe-addon \
  --cluster-name eks-cluster \
  --addon-name amazon-cloudwatch-observability \
  --query "addon.status" \
  --output text
```

{ Expected: Active }

11.Check CloudWatch Namespace

1. kubectl get namespace

12.Check CloudWatch Resources

1. kubectl get all -n amazon-cloudwatch

13.Check CloudWatch Agent DaemonSet

1. kubectl get daemonset -n amazon-cloudwatch

14.Check CloudWatch Agent Pods

```
kubectl get pods \
  -n amazon-cloudwatch \
  -l app.kubernetes.io/name=cloudwatch-agent \
  -o wide
```

15.Check CloudWatch Agent Daemonset

```
kubectl get daemonset \
  -n amazon-cloudwatch \
  -o wide
```

16.Check CloudWatch Agent logs

1. kubectl get pods -n amazon-cloudwatch

17.Check Fluent Bit

1. kubectl get pods -n amazon-cloudwatch -o wide

18.Check CloudWatch Console

1.Check Metrics

{ aws-console ---> CloudWatch ---> metrics ---> container Insights }

2.Check logs

{ aws-console ---> CloudWatch ---> logs ---> container Insights }

DEPLOYMENTS

Deployment in Kubernetes is an object used to manage, update, scale, and maintain application pods automatically.

It helps In

- create pods
- update pods
- scale pods
- recover failed pods automatically

Why We Use Deployment

1. Maintain Desired Number of Pods
2. Easy Application Updates (using multiple deployment strategies)
3. Scaling Applications (increasing or decreasing pods easily)
4. Rollback Support (restores the previous working version)

ROLLING OUT

1. Updating the application version gradually without stopping the entire application.
2. Kubernetes replaces old pods with new pods step-by-step.

Commands:

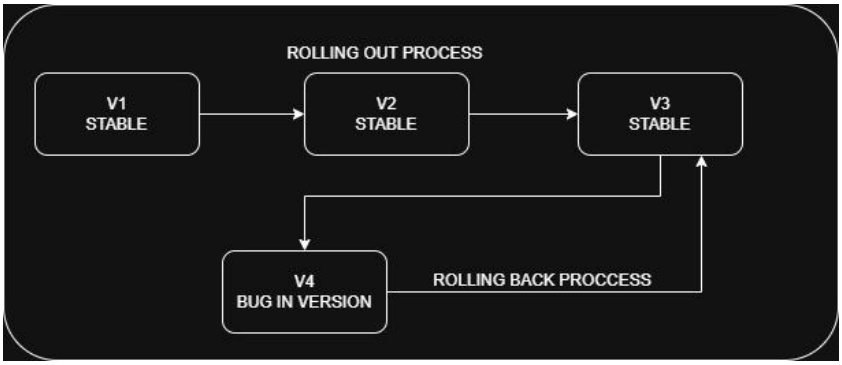
1. kubectl set image deployment/mydeploy nginx=nginx:1.25
2. kubectl rollout status deployment/mydeploy

ROLLING BACK

1. Going back to the previous working version if the new update fails.

Commands:

1. kubectl rollout history deployment/mydeploy
2. kubectl rollout undo deployment/mydeploy --to-revision=1



DEPLOYMENT STRATEGIES

1. Recreate Strategy

1. Old version of the application is completely terminated first, then the new version is deployed
2. Causes downtime because the old application stops before the new one starts.

   OLD VERSION → STOPPED
   NEW VERSION → STARTED

2. Rolling Update Strategy

1. New version is released gradually by replacing old pods one by one.
2. These strategy gets Zero or minimal downtime
3. Safe deployment strategy.

   OLD POD → NEW POD
   OLD POD → NEW POD

3. Blue-Green Deployment

1. release new version alongside with the old version then switch full traffic
2. Traffic is switched to the new version after testing.
3. Provides instant rollback capability.
4. Near zero downtime deployment.

   BLUE → OLD VERSION
   GREEN → NEW VERSION
   {traffic switches to green}

4. Canary Deployment

1. New version is released to a small percentage of users first.
2. Reduces deployment risk
3. Allows real user testing before full rollout.

   90% USERS → OLD VERSION
   10% USERS → NEW VERSION

5. A/B Testing Deployment

1. Different versions are released to different user groups for testing behavior and performance.

   GROUP A → VERSION A
   GROUP B → VERSION B

1. vim deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  labels:
    app: myapp
    env: devops
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      name: my-deployment-pods
      labels:
        app: myapp
    spec:
      containers:
        - name: nginx-cont
          image: nginx:latest
          ports:
            - name: myport
              containerPort: 80
```

2. kubectl apply -f deployment.yaml
3. kubectl get pods

ROLL-OUT NEW VERSION

1. kubectl set image deployment my-deployment nginx-cont=httpd:latest
2. watch kubectl get pods

ROLL-BACK TO OLD VERSION

1. kubectl rollout status deployment/my-deployment
2. kubectl rollout history deployment/my-deployment
3. kubectl rollout undo deployment/my-deployment

4. kubectl rollout undo deployment/my-deployment --to-revision=1
   {roll-back to specific version}
5. watch kubectl get pods

Note:

maxSurge

Extra pods allowed during update.

1. replicas = 4
2. maxSurge = 1
3. 4 running pods + 1 extra temporary pods = 5 pods maximum
4. Because Kubernetes first creates a new pod before deleting an old pod.
5. This keeps the application available with minimum downtime.

maxUnavailable

It simply means how many old pods can be unavailable (down) during a rolling update.

1. replicas = 4
2. maxUnavailable = 1
3. Kubernetes always keeps at least 3 pods running
4. Only 1 pod can go down at a time during update

STATEFULSETS

StatefulSet is a Kubernetes object used to deploy and manage stateful applications like databases, where pod identity, ordered deployment, and persistent storage are important.

FEATURES OF STATEFULSETS

1. Pods are created one by one in sequence like
    1. mysql-0
    2. mysql-1
    3. mysql-2
2. Pod names do not change even after restart.
3. Each pod gets its own persistent volume.
4. Pods are deleted in reverse order like
    1. mysql-2
    2. mysql-1
    3. mysql-0

THERE ARE TWO TYPES OF APPLICATIONS

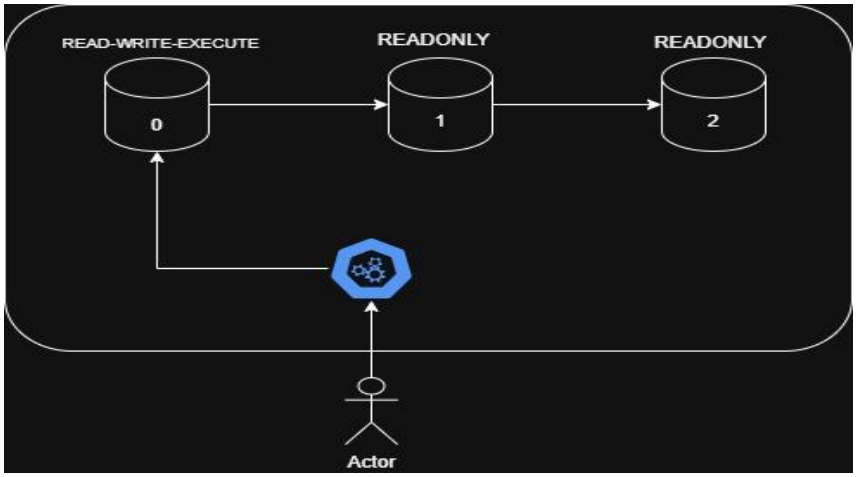
1. stateful applications
2. stateless applications

1. Stateful Applications

1. A stateful application is an application that stores and remembers data or session information between requests.
2. The application's state (data, user session, transactions, etc.) is important for the client.
3. Example:
    1. If a database pod is restarted and its data is lost, the client is affected.
    2. So databases are stateful applications.

2. Stateless Applications

1. A stateless application is an application that does not store client session data between requests.
2. Every request is treated as a new request.
3. Example:
    1. If one frontend pod stops and another pod starts, the client usually does not notice any difference.
    2. So frontend/API applications are mostly stateless applications.



1. vim statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-ss
  labels:
```

```
    app: myapp
    env: devops
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
        env: devops
    spec:
      containers:
      - name: mysql-container
        image: mysql:8.0
        ports:
        - name: mysql-port
          containerPort: 3306
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: "root123"
        - name: MYSQL_USER
          value: "admin"
        - name: MYSQL_PASSWORD
          value: "admin123"
        - name: MYSQL_DATABASE
          value: "mydb"
```

2. kubectl apply -f statefulset.yaml
3. kubectl get pods

STATEFULSET WITH HEADLESS SERVICE

1. vim ssheadless.yaml

```
apiVersion: apps/v1
```

```
kind: StatefulSet
metadata:
  name: my-ss
  labels:
    app: myapp
    env: devops
spec:
  serviceName: mysql-service
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
        env: devops
    spec:
      containers:
      - name: mysql-cont
        image: mysql:8.0
        ports:
        - name: mysql-port
          containerPort: 3306
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: "root123"
        - name: MYSQL_USER
          value: "admin"
        - name: MYSQL_PASSWORD
          value: "admin123"
        - name: MYSQL_DATABASE
          value: "mydb"
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: mysql-service

spec:
  clusterIP: None
  selector:
    app: myapp

  ports:
  - port: 3306
    targetPort: 3306
```

2. kubectl apply -f ssstatefulset.yaml
3. kubectl get pods

HEADLESS SERVICE

A Headless Service in Kubernetes is a service that does not give a single IP address to the pods.

Why we use headless service

We use Headless Service because StatefulSet pods need direct communication with specific pods, while ClusterIP sends traffic randomly.

EXAMPLE

Suppose you have a MySQL StatefulSet in Kubernetes:

1. mysql-0
2. mysql-1
3. mysql-2

Where:

- mysql-0 = Primary database
- mysql-1 = Replica
- mysql-2 = Replica

1. If You Use Normal ClusterIP Service

kind: Service
name: mysql-service

Now replicas connect like this:
mysql-1 → mysql-service

BUT ClusterIP does load balancing.
So Kubernetes may send request to:

- mysql-0 ✅
- mysql-1 ❌
- mysql-2 ❌

This is a problem because replica must sync data only from primary database.

2. If You Use Headless Service

Now Kubernetes creates DNS for every pod:

1. mysql-0.mysql-service
2. mysql-1.mysql-service
3. mysql-2.mysql-service

Now replica directly connects:

1. mysql-1 → mysql-0.mysql-service
2. mysql-2 → mysql-0.mysql-service

Now:

- No random routing
- No load balancing
- Direct pod communication

This is why StatefulSet uses Headless Service.
